

Приватний заклад вищої освіти
Одеський технологічний університет «ШАГ»
Кафедра інформаційних технологій та фундаментальної підготовки

випускна кваліфікаційна робота бакалавра

**Інформаційна платформа розповсюдження мультимедійного контенту за
моделлю підписки (на прикладі сервісу Bazinga)**

на здобуття бакалаврського рівня вищої освіти
зі спеціальності «122 Комп'ютерні науки»

Виконавці проекту:

№	П.І.Б.	Ролі	Група
1	[Прізвище Ім'я По батькові автора 1]	Full-Stack	КН-П-XXX
2	[Прізвище Ім'я По батькові автора 2]	Full-Stack	КН-П-XXX

Автор звіту: [Прізвище Ім'я По батькові автора 1]

Керів- ник	Доцент кафедри ІТ та ФП, к.т.н., [Прізвище І. П. керівни- ка]
---------------	--

Одеса – 2025

АНОТАЦІЯ

[Прізвище І. П. автора 1]. Інформаційна платформа розповсюдження мультимедійного контенту за моделлю підписки (на прикладі сервісу Bazinga). Підсистема облікових записів, профілів та платіжно-підписної моделі. — випускна кваліфікаційна робота на здобуття бакалаврського рівня вищої освіти зі спеціальності «122 Комп'ютерні науки». — Одеський технологічний університет «ШАГ», Одеса, 2025.

Робота присвячена проектуванню та повноцінній реалізації підсистеми облікових записів, профілів користувача та платіжно-підписного циклу для розподіленої інформаційної платформи Bazinga, що поєднує у собі сервіс для читання коміксів та манги (Bazinga Comics) і сервіс потокового перегляду відеоконтенту (BazingaTV). Виконавцем як Full-Stack розробником у межах роботи здійснено як проектування серверної частини відповідних модулів (на основі ASP.NET Core 9 та Entity Framework Core поверх MySQL), так і реалізацію клієнтської частини (на основі React 18, TypeScript та Vite з використанням бібліотеки компонентів shadcn/ui над Radix UI).

Актуальність роботи визначається стрімким розвитком стрімінгових та цифрових сервісів за моделлю «підписка як спосіб монетизації», які формують основний ринковий сегмент медіадистрибуції останніх років. Сучасна архітектура подібних платформ вимагає надійного, безпечного та масштабованого механізму реєстрації, керування багатoproфільним обліковим записом (за зразком Netflix), а також прозорого циклу взаємодії з платіжним шлюзом для оформлення підписок та збереження платіжних інструментів.

Об'єктом дослідження виступає підсистема ідентифікації, авторизації та платіжно-підписного обслуговування користувача в межах багаторівневої вебсистеми, побудованої за архітектурним стилем SPA + REST API. Предметом дослідження є методи безпечної реєстрації за магічним посиланням (passwordless email link sign-up), механізми багатoproфільного управління акаунтом, інтеграція з платіжним шлюзом Stripe в режимі test-mode, а також узгодження стану між клієнтом і сервером засобами JWT-автентифікації.

У ході роботи виконано такі задачі:

- проаналізовано сучасні підходи до проектування підсистем ідентифікації користувача (passwordless flow, magic-link, OAuth-аналоги) та обрано модель, найбільш придатну до сервісу медіадистрибуції;
- спроектовано схему бази даних для зберігання користувачів, профілів та одноразових токенів реєстрації; реалізовано механізм ідемпотентного оновлення схеми (CREATE TABLE IF NOT EXISTS) для розгортання на існуючих базах без втрати даних;

- реалізовано серверну частину: контролери `AuthController`, `ProfilesController`, `SubscriptionsController`, інфраструктурні сервіси `IMailSender` / `SmtplibEmailSender` (на основі бібліотеки `MailKit`), `IBillingService` / `StripeBillingService` (на основі `Stripe.NET`), а також сервіси `IJwtService` та `IPasswordHasher` для криптографічно стійкої обробки облікових даних;
- реалізовано клієнтську частину: посадкову сторінку, форму входу, чотирикрокового майстра реєстрації, селектор профілю, редактор профілю, сторінку керування профілями, сторінку особистого кабінету з боковою навігацією; забезпечено перемикання теми (світла / темна) на основі бібліотеки `next-themes`;
- забезпечено уніфікацію стану авторизації засобами `React Context` (`AuthContext`) із поділом сховищ за вимогою терміну дії: `localStorage` для довгострокових даних користувача та `sessionStorage` для активного профілю, що автоматично повертає прохання обрати профіль при кожному новому відвідуванні;
- реалізовано охоронні компоненти маршрутизації (`RequireAuth`, `RequireProfile`), які запобігають витоку контенту незареєстрованим користувачам та коректно опрацьовують стан очікування завантаження профілів при оновленні сторінки;
- інтегровано платіжний шлюз `Stripe` в режимі тестових ключів: серверний бік створює `Customer` та `SetupIntent`, клієнтський — використовує бібліотеку `@stripe/react-stripe-js` із компонентом `<CardElement />` для безпечного збору даних картки. Для випадку, коли ключі `Stripe` не сконфігуровані, реалізовано fallback на форму-симулятор, що дозволяє демонструвати потік без реальної інтеграції;
- реалізовано сервіс надсилання електронної пошти з підтримкою справжнього SMTP (`Gmail`, `Brevo`, `SendGrid` тощо) та fallback-режимом, який друкує згенероване магічне посилання у журнал — це забезпечує відтворюваність flow реєстрації у середовищах розробки без поштового сервера;
- здійснено інтеграційне тестування у браузерях `Chrome`, `Firefox` та `Edge`, перевірено перебіг повних сценаріїв «реєстрація → підтвердження пошти → вибір тарифу → введення картки → обрання профілю → перехід до основного контенту».

Результатом виконаної роботи є завершена, відмовостійка та підтримувана підсистема облікових записів та платіжно-підписного циклу, що задовольняє вимоги технічного завдання та готова до подальшої експлуатації у складі вебсистеми `Bazinga`.

ЗМІСТ

- ЗМІСТ
 - ВСТУП
 - ТЕХНІЧНЕ ЗАВДАННЯ ДО ПРОЄКТУ
 - РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ
 - РОЗДІЛ 2. РЕАЛІЗАЦІЯ ПІДСИСТЕМИ ОБЛІКОВИХ ЗАПИСІВ ТА ПРОФІЛІВ
 - РОЗДІЛ 3. РЕАЛІЗАЦІЯ ПЛАТІЖНО-ПІДПИСНОЇ МОДЕЛІ
 - ВИСНОВКИ
 - ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ
-

ВСТУП

Останнє десятиліття характеризується істотним зрушенням споживчого попиту у бік сервісів цифрової дистрибуції медіаконтенту, що працюють за моделлю «контент за підпискою» (subscription-on-content). Світовий ринок таких сервісів охоплює як вузькоспеціалізовані платформи для читання коміксів (Marvel Unlimited, ComiXology Unlimited, Shonen Jump+), так і універсальні стрімінгові сервіси загального призначення (Netflix, Disney+, Amazon Prime Video). Об'єднувальною ознакою цих сервісів є винесення на перший план таких функціональних блоків, як зручна реєстрація, прозорий цикл підписки, багатoproфільне використання облікового запису у межах однієї родини та персоналізована рекомендація контенту.

Технічна складність побудови подібних платформ зумовлюється трьома вимогами, які зазвичай вступають у компромісне співвідношення: безпекою (особисті та платіжні дані вимагають захисту відповідно до GDPR і PCI-DSS), швидкістю (час між кліком та відображенням контенту не повинен перевищувати декількох сотень мілісекунд), а також зручністю взаємодії (обмежена кількість кроків від лендингу до перегляду першого контенту). Якісно виконана підсистема реєстрації, профілю та підписки є передумовою всіх трьох названих вимог: вона визначає, чи стане потенційний користувач активним передплатником.

У межах цієї дипломної роботи виконавець як Full-Stack розробник реалізував підсистему облікових записів, профілів та платіжно-підписного циклу для гібридного сервісу Bazinga. Сервіс Bazinga поєднує у собі дві окремі частини: Bazinga Comics, яка надає доступ до читання коміксів та манги, та BazingaTV, яка дозволяє переглядати відеоконтент потоково. Спільна для обох частин підсистема, яка реалізована у роботі, охоплює увесь шлях користувача від заявки на реєстрацію через електронну пошту до моменту обрання активного профілю після успішного оформлення підписки.

Метою роботи є проєктування, реалізація та інтеграція повноцінної підсистеми керування обліковими записами, профілями користувача та платіжно-підписною моделлю для сервісу Bazinga, що відповідає сучасним вимогам безпеки, користувацького досвіду та масштабованості. Реалізація обіймає як серверну частину (ASP.NET Core, Entity Framework Core, MySQL), так і клієнтську (React, TypeScript, Tailwind CSS).

Завданнями дослідження є:

- аналіз сучасних підходів до passwordless-реєстрації та одно-/багатoproфільного облікового запису;
- проєктування схеми бази даних, програмних інтерфейсів та маршрутів навігації клієнта;

- реалізація серверних контролерів і сервісів, повний цикл інтеграції з платіжним шлюзом Stripe та поштовим сервером SMTP;
- реалізація клієнтських сторінок, форм та компонентів стану;
- забезпечення коректної взаємодії клієнт-серверних модулів засобами REST API + JWT-автентифікації;
- проведення інтеграційного тестування на охопленому функціоналі.

Робота побудована за патерном SPA + REST API: статичні ресурси клієнта обслуговуються NGINX, серверна частина — ASP.NET Core у контейнері Docker, дані зберігаються у MySQL.

ТЕХНІЧНЕ ЗАВДАННЯ ДО ПРОЄКТУ

Загальний опис

Програмне забезпечення (ПЗ) призначене для створення розподіленої інформаційної платформи Bazinga, що надає кінцевим користувачам можливість доступу до двох категорій контенту — коміксів/манги та потокового відео — за моделлю місячної підписки.

Архітектурно ПЗ ділиться на дві логічні половини. Перша половина — *Bazinga Comics* — забезпечує перегляд каталогу коміксів і манги, читання випусків онлайн (без можливості завантаження файлів локально), а також персоналізовану рекомендацію через систему фільтрів та інтеграцію з зовнішніми метаджерелами. Друга половина — *BazingaTV* — забезпечує потоковий перегляд відеоконтенту (мультсеріалів, аніме, ігрового відеоконтенту), агрегований через інтеграцію з відкритими метаджерелами на кшталт TVMaze.

Спільні елементи обох частин охоплюють:

- модель облікового запису з підтримкою до п'яти профілів на один обліковий запис (за зразком Netflix);
- модель підписки із трьома базовими тарифами — Bazinga Comics, BazingaTV та Bazinga Unlimited — і пробним семиденним тарифом, що автоматично переходить в один із оплачених при успішній прив'язці платіжного інструменту;
- адаптивний дизайн із підтримкою світлої і темної теми;
- універсальний пошук, що працює одночасно за коміксами, мангою та персонажами.

Призначення

ПЗ призначене для забезпечення зручної взаємодії між кінцевим користувачем (передплатником) та цифровим контентом, поданим у вигляді коміксів та потокового відео. Підсистема ідентифікації, профілів і підписок, реалізована у межах цієї роботи, призначена для виконання таких користувацьких сценаріїв:

- реєстрації нового користувача через електронну пошту (passwordless flow з підтвердженням посиланням);
- обрання тарифного плану підписки і прив'язки платіжного інструменту;
- управління обліковим записом: зміна імені, паролю, електронної пошти, перегляд інформації про підписку, керування платіжним інструментом, видалення облікового запису;
- створення, редагування та видалення профілів у межах облікового запису;
- обрання активного профілю при кожному новому відвідуванні;

- безпечного завершення сесії (logout) та повторного входу.

Функціональні вимоги до підсистеми ідентифікації, профілів та підписок

Реєстрація та автентифікація:

- реєстрація користувача починається з лендингу: користувач вводить електронну пошту та натискає «Get Started»;
- система виконує миттєву перевірку, чи зайнятий введений e-mail. Якщо так — перенаправляє на сторінку входу з підставленою адресою. Якщо ні — переходить на сторінку «Review to continue» зі згодою на маркетингові розсилки;
- після підтвердження користувач переходить на сторінку «Check your email», де очікує лист з посиланням-токеном;
- посилання дійсне 15 хвилин і має одноразовий характер; після переходу за посиланням користувач потрапляє на сторінку «Finish sign-up», яка реалізована як чотирикроковий майстер: Welcome → Password → Plan → Card;
- автентифікація постійних користувачів реалізована стандартним способом — e-mail + пароль; токен JWT (RFC 7519) формується сервером і зберігається у `localStorage` клієнта;
- передбачено механізм виходу з облікового запису (logout), що очищує локальне сховище та обнуляє активний профіль.

Профілі:

- один обліковий запис може містити до п'яти профілів;
- перший профіль (root) створюється автоматично при першому зверненні до `/api/profiles`;
- кожен профіль зберігає ім'я, аватар (колір + іконка-емодзі або URL), статус «Kids» (з відповідною фільтрацією контенту), приналежність до облікового запису;
- додавати та видаляти профілі може лише власник root-профілю; інші профілі можуть редагувати лише власні дані;
- після успішного входу або реєстрації користувач обов'язково проходить через сторінку «Who's watching?» для обрання активного профілю;
- активний профіль зберігається в `sessionStorage`, що забезпечує повторне прохання обрання при кожному новому відвідуванні (через закриття браузера).

Підписка та оплата:

- доступні тарифи: Bazinga Comics (\$7.99/mic), BazingaTV (\$9.99/mic), Bazinga Unlimited (\$14.99/mic);

- пробний 7-денний тариф Trial з повним доступом і автоматичним переходом на обраний оплачений тариф;
- після завершення Trial без вибору тарифу — доступ до контенту блокується, користувач спрямовується на сторінку обрання плану;
- ввід даних картки — лише через офіційний компонент Stripe Elements (PCI-DSS scope-out);
- для середовища без сконфігурованих ключів Stripe — fallback на форму-симулятор, яка дозволяє завершити демонстраційний flow без реальної інтеграції.

Особистий кабінет:

- сторінка «Account» з боковою навігацією, реалізована за зразком Netflix: розділи Overview, Subscription, Security, Devices, Profiles;
- редагування ПІБ, дати народження, e-mail, паролю;
- перегляд інформації про поточний тариф, дату наступного списання, останні чотири цифри картки;
- перемикання теми (світла / темна);
- безпечне видалення облікового запису.

Нефункціональні вимоги

- час відповіді API на ендпоінти автентифікації не повинен перевищувати 300 мс при середньому навантаженні;
- усі паролі зберігаються виключно у вигляді BCrypt-хешу (RFC 2898 / OWASP-сумісно);
- JWT-токен підписується HMAC SHA-256 з ключем, що береться з конфігурації або з env-змінної APP_SECURITY_JWT_SECRET;
- одноразові токени реєстрації зберігаються у вигляді SHA-256 хешу — оригінальне значення посилання після генерації не зберігається;
- бекенд гарантує ідемпотентну міграцію схеми (CREATE TABLE IF NOT EXISTS) — не вимагається ручних дій DevOps при оновленні існуючих БД;
- усі вихідні запити до Stripe огорнуто у тайм-аут (12 секунд) та механізм одного автоматичного повтору, що запобігає зависанню майстра реєстрації при недоступності api.stripe.com;
- клієнтська частина адаптивна — підтримує екрани від 320 px (мобільний) до 4K (десктоп);
- підтримуються браузері Chrome, Firefox, Safari, Edge у двох останніх стабільних версіях.

Архітектура

ПЗ спроектовано за клієнт-серверною моделлю з чітко відокремленими структурними вузлами:

- *Backend* — ASP.NET Core 9 Web API, що працює як stateless-сервіс і експонує REST-ендпоінти під префіксом `/api`;
- *Frontend* — SPA на React 18 + TypeScript, що збирається через Vite та обслуговується NGINX;
- *Database* — MySQL (через Pomelo.EntityFrameworkCore.MySql) у контейнері Docker;
- *Auxiliary services* — SMTP-сервер (для надсилання магічних посилань) та Stripe API (для платежів).

Технології

- Backend: C# 12, ASP.NET Core 9, Entity Framework Core 9, Pomelo MySQL 9.0, JWT (Microsoft.AspNetCore.Authentication.JwtBearer), BCrypt.Net-Next, MailKit, Stripe.net;
 - Frontend: TypeScript 5.8, React 18.3, Vite 5, Tailwind CSS 3, Radix UI / shadcn/ui, TanStack Query 5, React Router 6, next-themes, @stripe/stripe-js, @stripe/react-stripe-js;
 - Інфраструктура: Docker, Docker Compose, NGINX.
-

РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ

У ході виконання дипломної роботи команда з двох Full-Stack розробників створила повнофункціональну веб-платформу Bazinga, що поєднує у собі сервіс читання коміксів і манги (*Bazinga Comics*) та сервіс потокового перегляду відеоконтенту (*BazingaTV*). Платформа призначена для роботи у моделі підписки і охоплює весь шлях користувача — від першого знайомства зі стартовою сторінкою до повноцінного перегляду контенту в рамках обраного тарифного плану.

Архітектурно система побудована за моделлю Single Page Application (SPA) з вивокремленим бекендом, доступним через REST API. Цей підхід обрано як такий, що забезпечує найкращий компроміс між швидкістю відгуку інтерфейсу (за рахунок асинхронного клієнтського рендерингу без перезавантаження сторінки), гнучкістю масштабування (контейнеризований stateless-бекенд може горизонтально масштабуватися без обмежень) та зручністю розробки (чіткий поділ відповідальностей між клієнтом і сервером).

Серверна частина побудована на ASP.NET Core 9 з використанням Entity Framework Core поверх MySQL. Усі контролери побудовані відповідно до конвенцій REST: HTTP-методи GET / POST / PUT / DELETE використовуються згідно зі своїм семантичним значенням, відповіді мають стандартизовану форму JSON, а коди стану HTTP узгоджені з RFC 7231. Авторизація реалізована через JWT-токени (RFC 7519), підписані HMAC SHA-256.

Клієнтська частина побудована на React 18 у поєднанні з TypeScript та зібрана за допомогою Vite. Для стилізації використовується утиліторна CSS-бібліотека Tailwind CSS, а для уніфікованих компонентів інтерфейсу — бібліотека shadcn/ui, побудована поверх примітивів Radix UI. Управління серверним станом виконується через TanStack Query, що забезпечує автоматичне кешування, повторне завантаження та інвалідацію даних. Маршрутизація реалізована через React Router 6 із підтримкою захищених маршрутів.

Логічно платформа поділяється на дві великі підсистеми, що відповідають розподілу обов'язків між двома Full-Stack розробниками:

- 1. Підсистема облікових записів, профілів та підписок** (реалізована автором цього звіту) — охоплює реєстрацію, автентифікацію, керування багатопрофільним обліковим записом, тарифи підписки, інтеграцію з платіжним шлюзом Stripe та сервісом надсилання електронної пошти.

2. Підсистема каталогу контенту, метаданих та потокового відео (реалізована другим розробником) — охоплює відображення коміксів і манги, інтеграцію з відкритими метаджерелами (Jikan для аніме/манги, akabab/superhero-api для персонажів коміксів, TVMaze для серіалів), сторінку потокового перегляду, а також механізми пошуку та фільтрації.

Дві підсистеми взаємодіють між собою через спільний контекст автентифікації (`AuthContext` на стороні клієнта та `[Authorize]` атрибут на стороні сервера), а також через спільну схему бази даних, у якій таблиця `users` виступає центральним вузлом, на який спираються інші сутності.

Розгортання здійснюється засобами Docker та Docker Compose: окремі контейнери для бекенду, клієнта (NGINX, що віддає статичні файли збірки Vite) та бази даних. Це забезпечує переносимість між середовищами розробки і виробничим середовищем без необхідності ручного налаштування.

Розробка велася з використанням системи контролю версій Git із застосуванням практик pull request, код-рев'ю та інтеграційного тестування на гілці перед злиттям у головну. Усі компоненти, що потенційно змінюють базу даних, безпечно перевіряють схему за допомогою ідемпотентних `CREATE TABLE IF NOT EXISTS` блоків — це усуває ризик ручного дрейфу схеми при оновленні системи у виробничому середовищі.

РОЗДІЛ 2. РЕАЛІЗАЦІЯ ПІДСИСТЕМИ ОБЛІКОВИХ ЗАПИСІВ ТА ПРОФІЛІВ

2.1. Вибір технологій та підходів

Для серверної частини обрано фреймворк ASP.NET Core 9. Підставою вибору стала його сумісність з найновішим стандартом .NET, висока продуктивність порівняно з альтернативами (Node.js, Spring Boot) у бенчмарках TechEmpower, а також зрілість екосистеми бібліотек для типових завдань (JWT, EF Core, OpenAPI). Для роботи з базою даних обрано Entity Framework Core 9 у поєднанні з Pomelo.EntityFrameworkCore.MySql у preview-версії 9.0.0-preview.1, що забезпечує сумісність з MySQL та підтримку всіх потрібних функцій LINQ.

Для криптографічного хешування паролів обрано бібліотеку BCrypt.Net-Next: вона реалізує адаптивний алгоритм BCrypt з налаштовуваним фактором роботи, що залишається безпечним при еволюції апаратних потужностей, та сумісна з рекомендаціями OWASP.

Для генерації та валідації JWT-токенів використовується вбудована бібліотека `Microsoft.AspNetCore.Authentication.JwtBearer`. Токени підписуються HMAC SHA-256 з симетричним ключем, що береться з конфігураційного файлу або з env-змінної `APP_SECURITY_JWT_SECRET` (з пріоритетом env-змінної).

Для надсилання електронної пошти обрано бібліотеку MailKit 4.8 — це фактичний стандарт у .NET-екосистемі для роботи з SMTP, що підтримує STARTTLS, повну SSL-обгортку та аутентифікацію через будь-яку базову схему (LOGIN, PLAIN, XOAUTH2).

Для інтеграції з платіжним шлюзом обрано Stripe.NET 47.0 — це офіційна бібліотека-клієнт Stripe для .NET, що покриває всі поточні ендпоінти Stripe API.

Для клієнтської частини обрано React 18 + TypeScript 5.8 + Vite 5. React забезпечує компонентну модель та віртуальний DOM; TypeScript додає статичну типізацію, яка істотно знижує кількість помилок у часі компіляції; Vite надає швидку розробку через ESM-based hot reload та оптимізовану production-збірку. Tailwind CSS 3 використовується для стилізації, а shadcn/ui — як набір типових компонентів інтерфейсу (кнопки, форми, діалоги, спливаючі меню, перемикачі тощо), побудованих поверх примітивів Radix UI.

2.2. Модель бази даних

У межах підсистеми облікових записів задіяні три основні сутності:

Сутність `User` містить ідентифікатор, ім'я користувача (унікальне), електронну пошту (унікальна), хеш паролю, опціональні поля для ПІБ та дати народження, роль (`USER` або `ADMIN`), тип підписки та дату її закінчення, а також службові поля `CreatedAt` / `UpdatedAt`. EF Core встановлює унікальні індекси на колонки `username` та `email` для забезпечення цілісності даних.

Сутність `Profile` містить ідентифікатор, посилання на власника (`UserId`), ім'я профілю, опціональні URL аватара та колір/іконку для замітника, прапорці `IsRoot` (вказує, чи це головний профіль облікового запису) та `IsKids` (визначає, чи слід обмежувати контент за віком). Зв'язок з `User` встановлений каскадно: при видаленні користувача автоматично видаляються всі його профілі.

Сутність `SignupToken` містить ідентифікатор, електронну пошту, до якої прив'язано токен, SHA-256 хеш самого значення токена, дату закінчення дії (`ExpiresAt` = час створення + 15 хв), дату використання (`ConsumedAt`, nullable), а також прапорець відмови від маркетингових розсилок. Оригінальне значення токена ніколи не зберігається — у БД потрапляє лише хеш, що унеможливлює доступ до активних посилань навіть при компрометації БД.

Для забезпечення ідемпотентного оновлення схеми у `Program.cs` реалізовано виклик `ExecuteSqlRaw` із `CREATE TABLE IF NOT EXISTS` для всіх трьох таблиць. Цей блок виконується одразу після `EnsureCreated()` і дозволяє оновлювати екземпляри платформи, які вже працювали з попередньою версією схеми, без втрати даних та без необхідності ручного втручання адміністратора.

2.3. Серверні контролери

`AuthController` забезпечує усі ендпоінти автентифікації та реєстрації:

- `POST /api/auth/register` — класична реєстрація з логіном і паролем (зарезервовано для адмін-сценарію);
- `POST /api/auth/login` — вхід за e-mail/іменем користувача та паролем; повертає JWT-токен і профіль користувача;
- `POST /api/auth/signup/check` — швидка перевірка зайнятості e-mail (використовується лендингом для миттєвої реакції на введення);
- `POST /api/auth/signup/start` — генерує одноразовий токен реєстрації, надсилає на e-mail магічне посилання вигляду `/signup/complete?token=...`; перед цим інвалідує всі попередні активні токени для цього e-mail;
- `GET /api/auth/signup/verify?token=...` — перевіряє валідність токена (існує / не використано / не прострочено) і повертає прив'язану електронну пошту;

- `POST /api/auth/signup/billing-intent` — створює Stripe Customer і `SetupIntent`, повертає `clientSecret` для подальшого використання у клієнтських Stripe Elements; якщо Stripe не сконфігуровано, повертає `stripeConfigured: false`;
- `POST /api/auth/signup/complete` — приймає токен, пароль, обраний тариф і опціональний `paymentMethodId` зі Stripe; створює користувача, відмічає токен як використаний, повертає JWT.

ProfilesController реалізує повний CRUD над сутністю `Profile`:

- `GET /api/profiles` — список профілів поточного користувача; якщо профілів немає (наприклад, для тільки-но створеного облікового запису) — автоматично створює root-профіль з імені користувача;
- `POST /api/profiles` — створення нового профілю; перевіряє обмеження «не більше п'яти профілів»;
- `PUT /api/profiles/{id}` — оновлення; перевіряє, що профіль належить поточному користувачу;
- `DELETE /api/profiles/{id}` — видалення; забороняє видалення root-профілю (повертає 400);
- усі ендпоінти захищені атрибутом `[Authorize]` і використовують допоміжний клас `CurrentUser` для отримання поточного користувача з JWT-клеймів.

SubscriptionsController надає інформацію про поточний тариф підписки користувача та підтримує адміністративну зміну тарифу через `PUT /api/subscriptions/{userId}`.

2.4. Інфраструктурні сервіси

ISender / SmtpEmailSender реалізує надсилення електронної пошти через MailKit. Конфігурація береться з `EmailOptions` (хост, порт, ім'я користувача, пароль, ім'я відправника, e-mail відправника). Підтримуються як STARTTLS (для більшості SMTP-провайдерів на порту 587), так і повний SSL (порт 465). Реалізована також fallback-версія `LoggingEmailSender`, яка просто записує адресу одержувача, тему та згенероване посилання у журнал — це використовується у середовищах розробки, де SMTP-сервер не сконфігуровано, але потрібно імітувати весь flow реєстрації.

IBillingService / StripeBillingService забезпечує інтеграцію зі Stripe. Конструктор бере секретний і публічний ключі з `StripeOptions` (або з env-змінних `STRIPE_SECRET_KEY` / `STRIPE_PUBLISHABLE_KEY`) і налаштовує `StripeClient` з `HttpClient`-обгорткою, що має таймаут 12 секунд та один автоматичний повтор. Метод `CreateSetupIntentAsync` створює об'єкт `Customer` у Stripe (з прив'язкою e-mail) та `SetupIntent` для збереження картки до подальшого використання (use case

off_session). Контролер додатково обгортає виклик try/catch блоком, що повертає 502 при StripeException та 504 при таймауті — це усуває ризик зависання клієнта при недоступності api.stripe.com.

IJwtService / **JwtService** інкапсулює генерацію JWT-токенів. До токена додаються клейми sub (e-mail), nameidentifier (id користувача), name (ім'я користувача) і role. Термін дії — ExpirationMs з конфігурації, типово одна година.

IPasswordHasher / **BCryptPasswordHasher** інкапсулює виклик BCrypt.HashPassword / BCrypt.Verify. Завдяки винесенню за інтерфейс зміна алгоритму хешування у майбутньому не вплине на код контролерів.

2.5. Клієнтська частина

Клієнтський потік реєстрації реалізовано як ланцюжок з чотирьох сторінок:

1. **Landing.tsx** — посадкова сторінка з полем введення e-mail. При відправці викликає POST /api/auth/signup/check. Якщо e-mail зайнятий — перенаправляє на /auth?mode=signin&email=... (форма входу з підставленим e-mail). Якщо вільний — перенаправляє на /signup/review?email=...
2. **SignUpReview.tsx** — сторінка «Review to continue»: відображає введений e-mail з можливістю змінити, відмітка про відмову від маркетингових розсилок, кнопка «Continue». На відправці викликає POST /api/auth/signup/start, після чого перенаправляє на /signup/check-email?email=...
3. **SignUpCheckEmail.tsx** — сторінка «Tap the link in your email»: інструктує користувача перевірити пошту, надає кнопку «Resend it» (повторно надсилає лист) та посилання на сторінку входу для тих, хто вже має акаунт. Для середовища розробки, де SMTP не налаштовано, на сторінці є дев-нотатка з підказкою шукати посилання у логах API.
4. **SignUpComplete.tsx** — фінальна сторінка-майстер, реалізована як кінцевий автомат з чотирма станами: - **Welcome** — користувачу пропонується два сценарії: «Explore for Free» (стартує 7-денний пробний тариф) або «Join Now» (одразу обирає платний); - **Password** — введення паролю з підтвердженням (мінімум 6 символів); - **Plan** (тільки для платного шляху) — обрання одного з трьох тарифів: Bazinga Comics, BazingaTV або Bazinga Unlimited; - **Card** — інтеграція з Stripe Elements через StripeCardForm.tsx; форма-симулятор для середовища без Stripe-ключів.

Майстер реалізує наскрізну навігацію Back-кнопками між станами, прогрес-індикатор з мітками виконаних кроків, а також автоматичну верифікацію магічного токена при завантаженні сторінки.

Після успішного завершення реєстрації автоматично виконується вхід (`AuthContext.completeSignup` оновлює стан, JWT зберігається у `localStorage`), і користувач перенаправляється на сторінку селектора профілів `/profiles`.

ProfileSelector.tsx реалізує сторінку «Who's watching?» за зразком Netflix: великі плитки з аватарами, що завантажуються асинхронно через `GET /api/profiles`. Активний профіль зберігається у `sessionStorage`, який автоматично очищується при закритті вкладки/браузера — це забезпечує повторне прохання обрання при кожному новому відвідуванні. Тільки root-профіль (або стан «профіль не обрано») має право створювати нові профілі.

ManageProfiles.tsx надає список усіх профілів з можливістю редагування та видалення; видалення підтверджується через діалогове вікно з компонента `AlertDialog`.

ProfileEditor.tsx реалізує форму редагування одного профілю: ім'я, палітра кольорів аватара, набір емодзі-іконок, URL аватара (опціонально), перемикач «Kids profile».

Profile.tsx реалізує особистий кабінет з боковою навігацією за зразком Netflix: розділи Overview, Subscription, Security, Devices, Profiles. В розділі Overview додатково розміщено перемикач теми (світла/темна), реалізований через бібліотеку `next-themes`.

Choice.tsx — сторінка обрання експерієнсу (Read vs Watch), що відкривається після обрання профілю. Вона ефективно є точкою «вилки» між двома підсистемами Bazinga: переходом у Comics або у BazingaTV.

2.6. Управління станом та маршрутизація

Управління станом автентифікації централізовано через React Context (`AuthContext`). У контексті зберігається: - `user` — об'єкт користувача (з `localStorage`); - `token` — JWT (з `localStorage`); - `profiles` — масив профілів (завантажується з API при наявності токена); - `currentProfile` — обчислюваний на основі `profiles` та `currentProfileId` з `sessionStorage`; - `profilesLoading` — прапорець завантаження; - допоміжні `hasPaidSubscription`, `trialExpired` — обчислюються з типу підписки.

Контекст також надає методи `login`, `register`, `completeSignup`, `logout`, `selectProfile`, `clearProfile`, `refreshProfiles`, `createProfile`, `saveProfile`, `removeProfile`.

Маршрутизація реалізована через React Router 6 із трьома рівнями захисту: - **RootGate** — корінь `/`: показує `Landing`, якщо користувач не залогований; `ProfileSelector`, якщо профіль не обрано; `Choice`, якщо все добре; - **RequireAuth**

— для маршрутів, що вимагають лише входу (`/profile`, `/profiles*`); - **RequireProfile** — для контентних маршрутів (`/comics`, `/bazinga-tv` тощо); додатково перенаправляє користувачів з простроченим Trial-тарифом на сторінку обрання плану.

Окремо реалізовано компонент `ScrollToTop`, що повертає вікно на верх при кожній навігації — це усуває типову незручність SPA, коли користувач при переході на нову сторінку залишається на тій же позиції прокручування, на якій був на попередній.

РОЗДІЛ 3. РЕАЛІЗАЦІЯ ПЛАТІЖНО-ПІДПИСНОЇ МОДЕЛІ

3.1. Архітектура інтеграції зі Stripe

Інтеграцію з платіжним шлюзом Stripe побудовано за принципом «PCI-DSS scope minimization»: чутливі дані картки (номер, термін дії, CVC) ніколи не потрапляють на сервер Bazinga. Ввід даних виконується виключно у компонентах Stripe Elements, які запускаються всередині iframe з домену `js.stripe.com` — формально кожен інстанс компонента є окремим контекстом виконання, недоступним для зовнішнього JavaScript.

Послідовність взаємодії така:

1. Клієнт відкриває крок Card майстра реєстрації;
2. Викликає `POST /api/auth/signup/billing-intent` з токеном магічного посилення;
3. Серверний `StripeBillingService` створює `Stripe Customer` (з прив'язкою e-mail) та `SetupIntent` для збереження картки;
4. Сервер повертає `{ clientSecret, customerId, publishableKey }`;
5. Клієнт викликає `loadStripe(publishableKey)` та обгортає форму в `<Elements stripe={stripePromise} options={{ clientSecret }}>`;
6. Користувач вводить картку у `<CardElement />`;
7. При натисканні «Submit» викликається `stripe.confirmCardSetup(clientSecret, { payment_method: { card: cardElement } })`;
8. Якщо успіх — отримуємо `payment_method.id`, який передаємо у `POST /api/auth/signup/complete` разом з токеном та паролем;
9. Сервер створює користувача з відповідним тарифом і повертає JWT.

Цей flow забезпечує, що єдиний контакт сервера з даними картки — це отримання `payment_method.id`, який є непрозорим ідентифікатором Stripe і не дозволяє отримати дані картки без виклику Stripe API з відомим секретним ключем.

3.2. Fallback-режим без Stripe

Для середовищ розробки та демонстрації, де може бути недоступним справжній обліковий запис Stripe, реалізовано fallback на mock-форму. Логіка така:

- сервер визначає, чи сконфігуровано Stripe, перевіряючи наявність `SecretKey` і `PublishableKey` у `StripeOptions`;
- якщо ключів немає — `POST /api/auth/signup/billing-intent` повертає `{ stripeConfigured: false }`;

- клієнт у цьому випадку рендерить власну форму з полями «Name», «Card number», «Expiry», «CVC», що мають базову маску вводу та валідацію;
- при відправці у `POST /api/auth/signup/complete` поле `paymentMethodId` залишається `null`;
- сервер створює користувача у звичайний спосіб, але без прив'язки до Stripe Customer.

Це дозволяє повноцінно демонструвати майстер реєстрації навіть без реального Stripe-облікового запису, що особливо корисно у середовищі захисту дипломної роботи.

3.3. Тарифна модель та автоматичне закінчення Trial

Реалізована тарифна модель базується на полі `SubscriptionType` сутності `User`, яке може приймати одне з чотирьох значень:

- `Trial` — пробний 7-денний тариф (присвоюється за замовчуванням при реєстрації через шлях «Explore for Free»); має повний доступ до контенту;
- `Comics` — Bazinga Comics (\$7.99/mic); включає тільки доступ до коміксів і манги;
- `TV` — BazingaTV (\$9.99/mic); включає тільки доступ до потокового відео;
- `Unlimited` — Bazinga Unlimited (\$14.99/mic); включає все.

Дата закінчення тарифу зберігається у полі `SubscriptionExpiration` (`DateOnly?`). Для оплачених тарифів — це поточна дата + 30 днів; для `Trial` — поточна + 7 днів.

Допоміжна функція `isTrialExpired` (на стороні клієнта) перевіряє, чи `SubscriptionType === "Trial"` та чи `SubscriptionExpiration < Date.now()`. Якщо так — `RequireProfile` автоматично перенаправляє користувача на сторінку обрання тарифу `/bazinga-unlimited?from=trial-expired`. Це гарантує, що `Trial`-користувач, який не оформив платну підписку до сьогомого дня, не зможе продовжувати користуватися контентом, але матиме можливість оформити підписку без втрати облікового запису та профілів.

Допоміжна функція `isPaidSubscription` визначає, чи є тариф оплаченим. Її результат використовується у компоненті `SiteHeader` для приховування кнопки «Join Now» від уже-підписаних користувачів — типова, але часто ігнорована проблема UX.

3.4. Інтеграція з підсистемою профілів

Підсистема профілів тісно інтегрована з підсистемою підписки через спільну авторизаційну модель. Кожен запит до `/api/profiles*` потребує валідного JWT-токена, що несе ідентифікатор користувача. Профілі належать користувачу і не мають окремих облікових даних — вибір профілю є виключно клієнтським станом, що зберігається у `sessionStorage`.

Це означає, що: - профіль не може бути використаний без активного входу в обліковий запис; - блокування облікового запису (наприклад, при простроченому Trial) автоматично блокує всі профілі; - зміна тарифу одразу впливає на всі профілі облікового запису.

Такий підхід відповідає типовій моделі «family plan» (один обліковий запис — кілька профілів), що використовується у Netflix, Spotify Family та подібних сервісах.

3.5. Особистий кабінет

Особистий кабінет (`Profile.tsx`) реалізовано як сторінку з боковою навігацією за зразком Netflix-account. Доступні розділи:

- **Overview** — основна інформація (ПІБ, дата народження), форма редагування, перемикач теми (світла/темна);
- **Subscription** — поточний тариф, дата наступного списання, можливість змінити план через `/bazinga-unlimited`;
- **Security** — пароль (приховано), e-mail, номер телефону (поки заглушка);
- **Devices** — список пристроїв; поточний пристрій з кнопкою «Sign out»;
- **Profiles** — короткий список профілів з посиланням на `/profiles/manage`.

Перемикач теми працює через бібліотеку `next-themes`, що додає клас `dark` до елемента `<html>` при ввімкненні темної теми. CSS-змінні у файлі `src/index.css` визначають окремі набори кольорів для світлого режиму (`:root`) та темного (`.dark`).

3.6. Захист від типових атак

Реалізовано такі заходи безпеки:

- *проти reverse-engineering пароля з БД* — BCrypt з адаптивним фактором роботи (типово 11);
- *проти крадіжки сесії* — JWT з обмеженим терміном дії (1 година) і підписом HMAC SHA-256;
- *проти підбору магічних посилань* — токени з 256-бітної ентропії (32 байти `RandomNumberGenerator`), збереження лише SHA-256 хешу;

- *проти повторного використання посилань* — поле `ConsumedAt`, що встановлюється при першому успішному `signup/complete`;
 - *проти спаму запитує `signup/start`* — повторні виклики для одного e-mail інвалідують усі попередні активні токени, лишаючи в активному стані лише найсвіжіший;
 - *проти XSS у даних користувача* — React за замовчуванням екранує всі рядкові значення в JSX;
 - *проти CSRF* — використання Bearer-токена у заголовку `Authorization` (а не у cookie), що унеможлиблює CSRF-атаки за дизайном.
-

ВИСНОВКИ

У ході виконання дипломної роботи спроектовано та повністю реалізовано підсистему облікових записів, профілів та платіжно-підписної моделі для гібридної інформаційної платформи Bazinga. Реалізація охоплює як серверну частину (ASP.NET Core 9 + Entity Framework Core + MySQL), так і клієнтську (React 18 + TypeScript + Vite + Tailwind CSS).

Серверна частина забезпечує повний цикл passwordless-реєстрації через магічне посилання у пошті, класичну автентифікацію за e-mail/паролем, керування багатопрофільним обліковим записом (до п'яти профілів на користувача), інтеграцію з платіжним шлюзом Stripe у режимі test-mode, а також надсилання електронної пошти через будь-який SMTP-сервер з можливістю fallback на запис у журнал.

Клієнтська частина реалізує посадкову сторінку, форму входу, чотирикроковий майстер реєстрації, селектор профілю, редактор профілю, сторінку керування профілями та особистий кабінет з боковою навігацією. Усі компоненти стилізовано засобами Tailwind CSS + shadcn/ui з підтримкою світлої та темної теми.

У ході реалізації виконано такі ключові завдання:

- спроектовано та реалізовано схему бази даних для зберігання користувачів, профілів та одноразових токенів реєстрації; забезпечено ідемпотентне оновлення схеми засобами `CREATE TABLE IF NOT EXISTS`;
- реалізовано чотири ендпоінти passwordless-реєстрації: перевірка зайнятості e-mail, ініціювання реєстрації, верифікація токена, фіналізація реєстрації;
- реалізовано повний CRUD над сутністю профілю з контролем обмеження «не більше п'яти профілів на акаунт» та заборonoю видалення root-профілю;
- реалізовано інтеграцію зі Stripe з мінімізацією PCI-DSS scope: дані картки ніколи не потрапляють на сервер Bazinga;
- реалізовано fallback-режим для середовищ без Stripe (повертає `stripeConfigured: false`, клієнт автоматично перемикається на mock-форму);
- реалізовано fallback-режим для середовищ без SMTP (магічне посилання записується у журнал замість надсилання листа);
- реалізовано клієнтський AuthContext з диференційованим зберіганням (localStorage для облікового запису, sessionStorage для активного профілю);
- реалізовано три рівні захисту маршрутизації: `RootGate`, `RequireAuth`, `RequireProfile`;

- реалізовано автоматичне перенаправлення Trial-користувачів з простроченим тарифом на сторінку обрання плану;
- реалізовано особистий кабінет за зразком Netflix-account з боковою навігацією та підрозділами Overview, Subscription, Security, Devices, Profiles;
- забезпечено підтримку світлої та темної теми через бібліотеку `next-themes` з повним переходом CSS-змінних;
- реалізовано допоміжний компонент `ScrollToTop` для коректного скидання позиції прокручування при кожній навігації;
- здійснено інтеграційне тестування у браузерях Chrome, Firefox, Edge для перебігу повних сценаріїв реєстрації, входу, обрання профілю та оновлення тарифу.

Особливу увагу приділено надійності функціонування у нестандартних умовах: при недоступності `api.stripe.com` — таймаут 12 секунд та fallback-форма; при недоступності SMTP — запис у журнал; при першому завантаженні сторінки до завершення асинхронного отримання списку профілів — відображення короткого стану «Loading your profile...» замість помилкового перенаправлення на селектор.

Обрані технології та архітектурні рішення повністю задовольнили вимоги технічного завдання. Підсистема є завершеною, працездатною, відмовостійкою та готовою до подальшого розвитку — як з боку додаткових функцій (наприклад, реальної інтеграції з відправкою СМС для двофакторної автентифікації), так і з боку масштабування (бекенд є stateless і може бути горизонтально масштабованим).

Подальшими напрямками розвитку можуть бути: впровадження двофакторної автентифікації (TOTP), інтеграція з зовнішніми OAuth-провайдерами (Google, Apple), реалізація PIN-захисту окремих профілів (особливо Kids), а також додавання повноцінного механізму відновлення паролю через магічне посилання.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

- ASP.NET Core Documentation — <https://learn.microsoft.com/aspnet/core>
- Entity Framework Core Documentation — <https://learn.microsoft.com/ef/core>
- Pomelo.EntityFrameworkCore.MySql — <https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql>
- React Documentation — <https://react.dev/learn>
- TypeScript Documentation — <https://www.typescriptlang.org/docs/>
- Vite Documentation — <https://vite.dev/guide/>
- Tailwind CSS Documentation — <https://tailwindcss.com/docs>
- shadcn/ui — <https://ui.shadcn.com/>
- Radix UI Primitives — <https://www.radix-ui.com/primitives>
- TanStack Query — <https://tanstack.com/query/latest>
- React Router 6 — <https://reactrouter.com/>
- next-themes — <https://github.com/pacocoursey/next-themes>
- Stripe API Reference — <https://stripe.com/docs/api>
- Stripe.NET — <https://github.com/stripe/stripe-dotnet>
- Stripe Elements (React) — <https://stripe.com/docs/stripe-js/react>
- MailKit — <https://github.com/jstedfast/MailKit>
- BCrypt.Net-Next — <https://github.com/BcryptNet/bcrypt.net>
- JWT RFC 7519 — <https://datatracker.ietf.org/doc/html/rfc7519>
- Bearer Token RFC 6750 — <https://datatracker.ietf.org/doc/html/rfc6750>
- BCrypt Password Hashing — <https://en.wikipedia.org/wiki/Bcrypt>
- OWASP Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Netflix-style Profiles UX Pattern — <https://help.netflix.com/en/node/10421>
- Репозиторій проекту — <https://github.com/dentuss/Bazinga>